

Puscasu Tudor & Duica Vlad

HOME

ABOUT

CONTENT

OTHERS

EventMate

Presentation

Plan events, invite friends, track RSVPs effortlessly



The Problem

Organizing social events through group chats is frustrating. Messages pile up, event details get buried, and nobody knows who's actually coming. You end up asking "so who's in?" multiple times, and there's no way to check plans when you're offline.

Social media has become less social

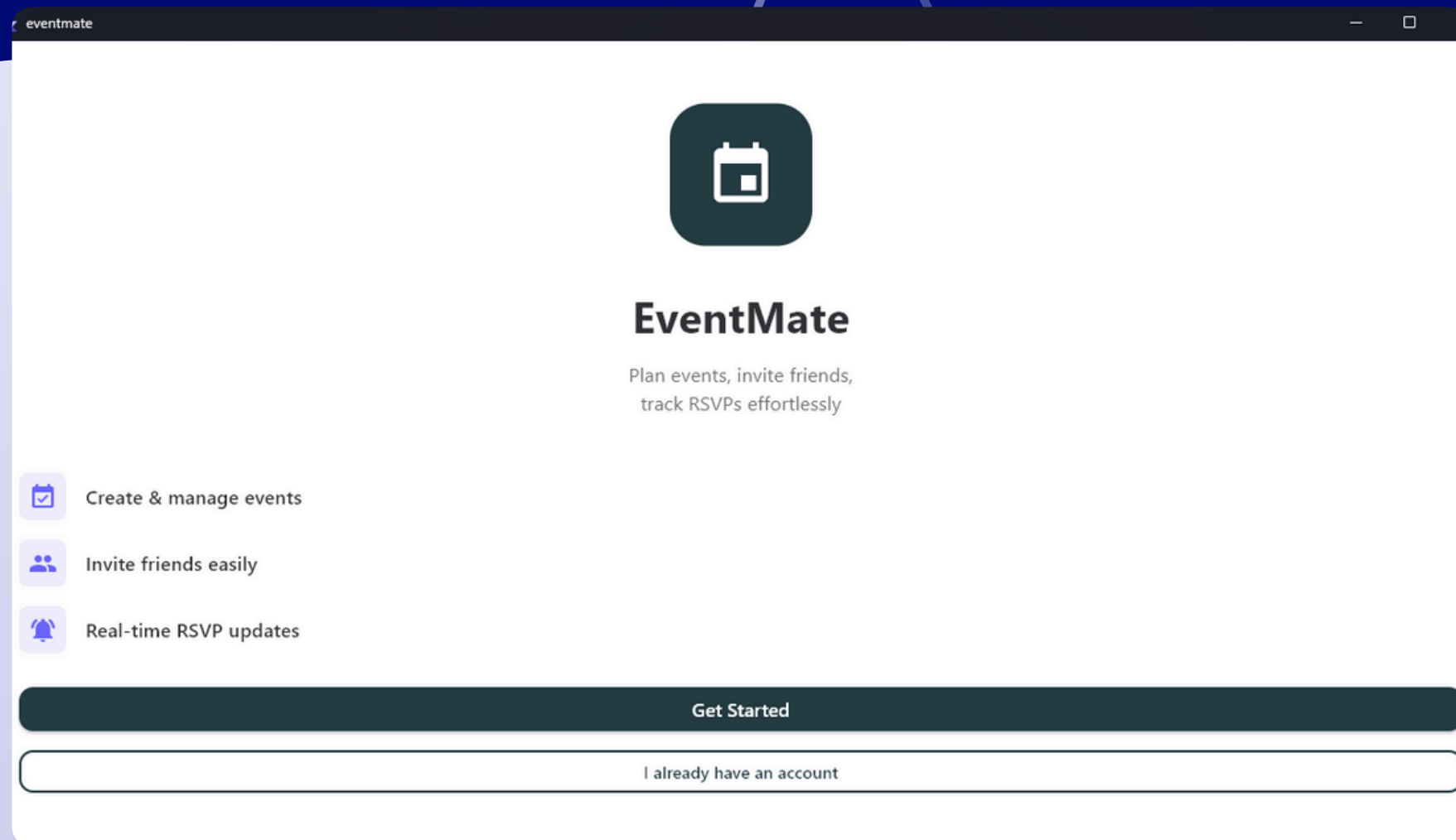
Change in share of people reporting each reason for using social media (%)



Source: [GWI](#)

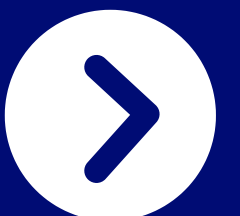
FT graphic: John Burn-Murdoch / @jburnmurdoch

©FT



Our Solution

EventMate is a dedicated mobile app that simplifies event planning. Create an event, invite your friends, and watch the RSVPs roll in. Everything stays organized in one place, and it works even without internet connection.



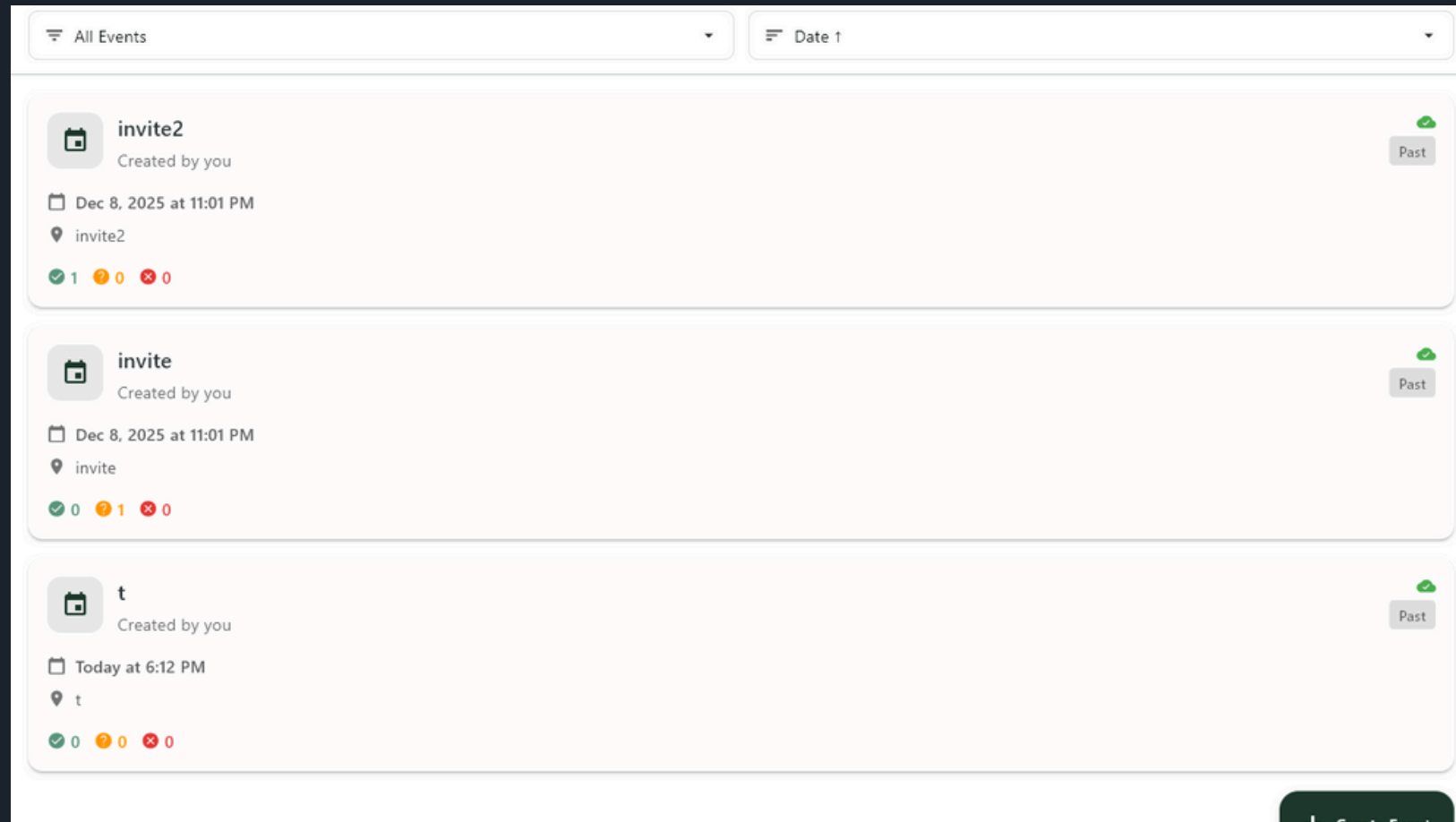
Core Features

Event Creation – Set up events with all the details: title, date, time, location, and description.

Invitations – Add friends by email and they'll be notified instantly.

RSVP Tracking – See who's Going, Maybe, or Can't Go at a glance.

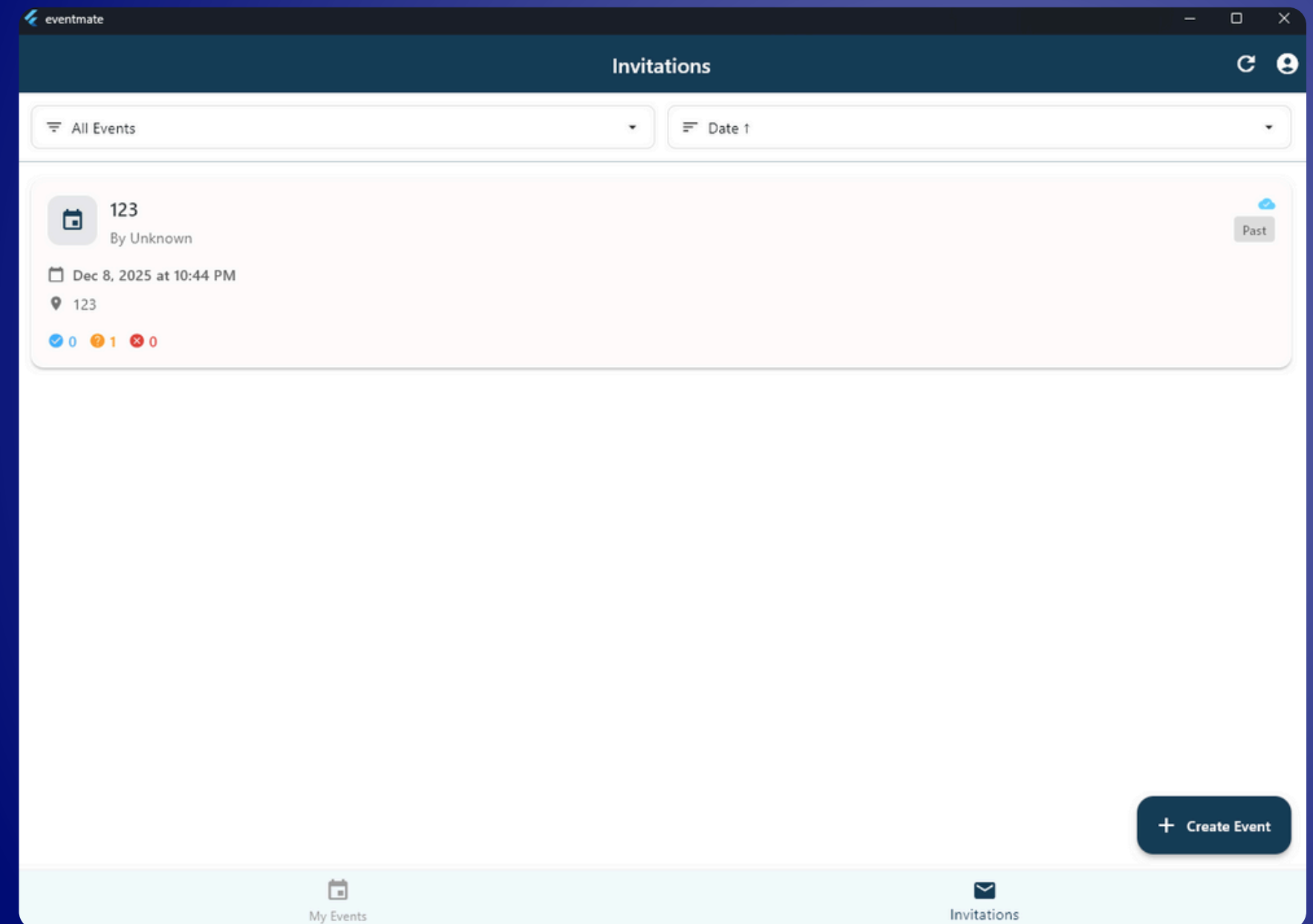
Offline Mode – Access your events anytime, even without internet.



App Walkthrough

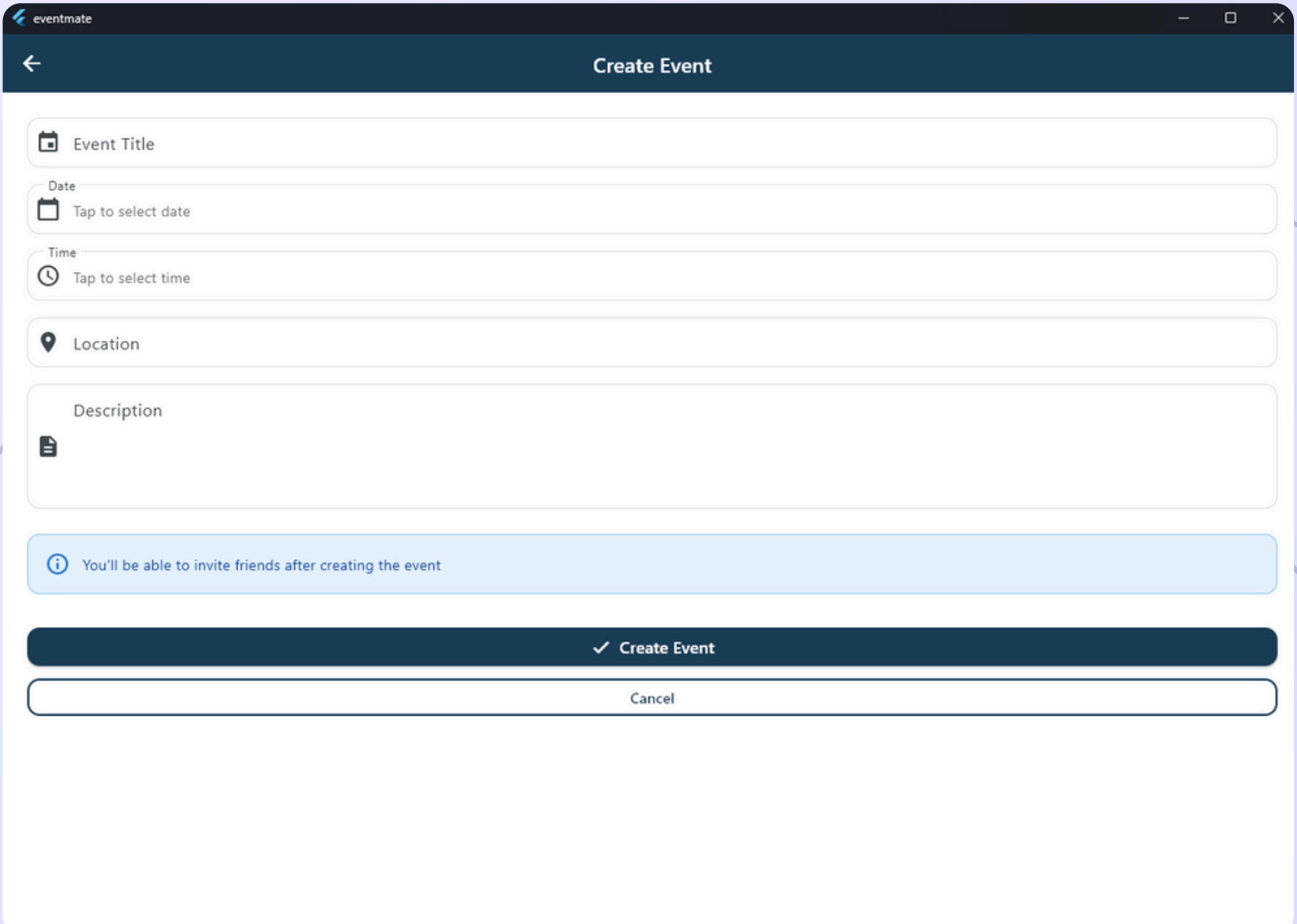
Think Social:

Users start at the welcome screen and either sign up or log in. The home screen shows two tabs: "My Events" for events you created, and "Invitations" for events you're invited to. A floating button lets you quickly create new events.



Creating Events Features

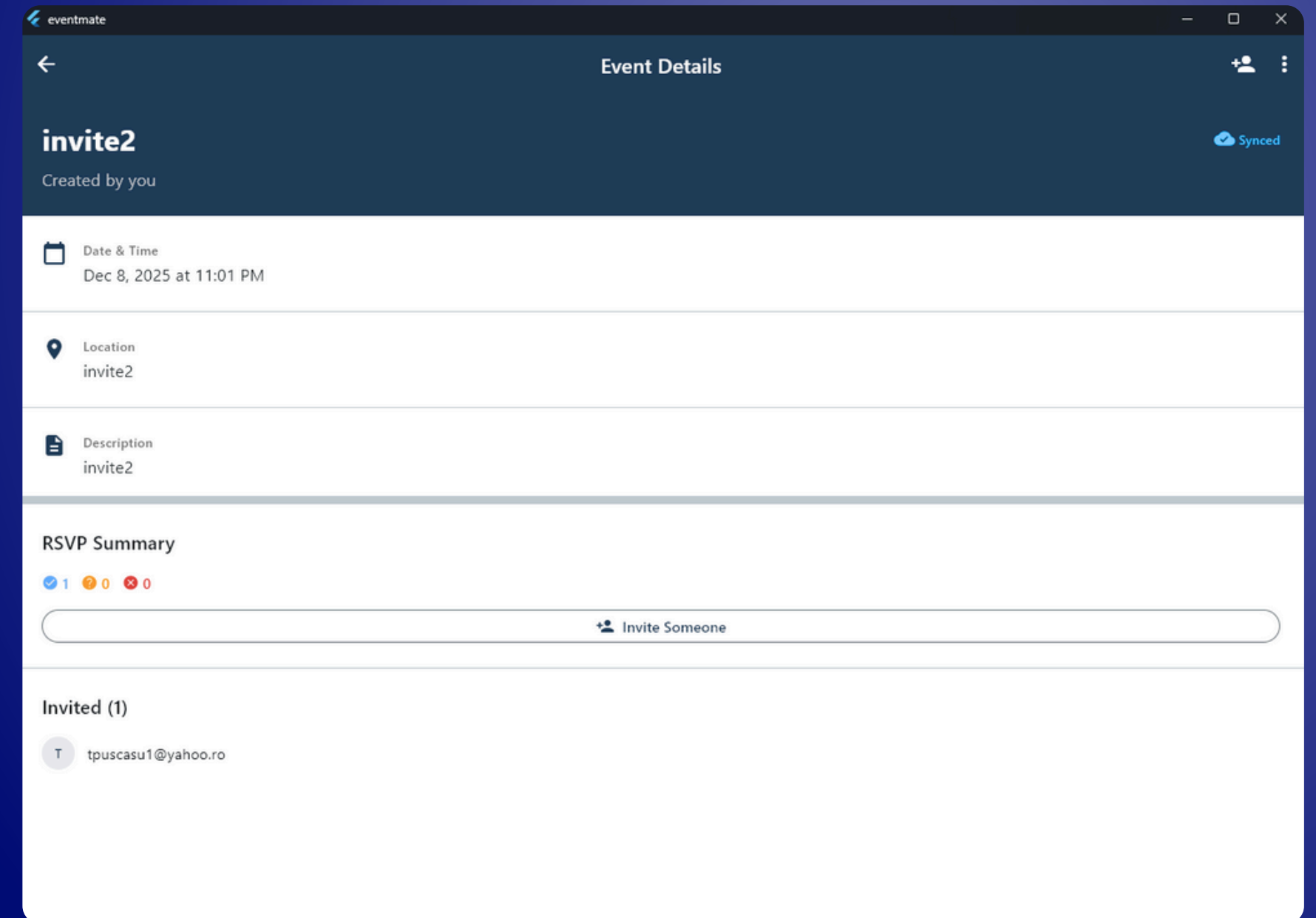
The event creation screen keeps things simple. Enter a title, pick the date and time using native pickers, add a location and description. You can also invite people right away by entering their emails.



The screenshot displays the 'Create Event' interface within the EventMate app. The screen features a dark blue header with a back arrow and the title 'Create Event'. Below the header, there are five input fields: 'Event Title', 'Date' (with a calendar icon and 'Tap to select date'), 'Time' (with a clock icon and 'Tap to select time'), 'Location' (with a location pin icon), and 'Description' (with a document icon). A light blue informational banner states: 'You'll be able to invite friends after creating the event'. At the bottom, there are two buttons: a dark blue 'Create Event' button with a checkmark icon, and a white 'Cancel' button with a dark border.

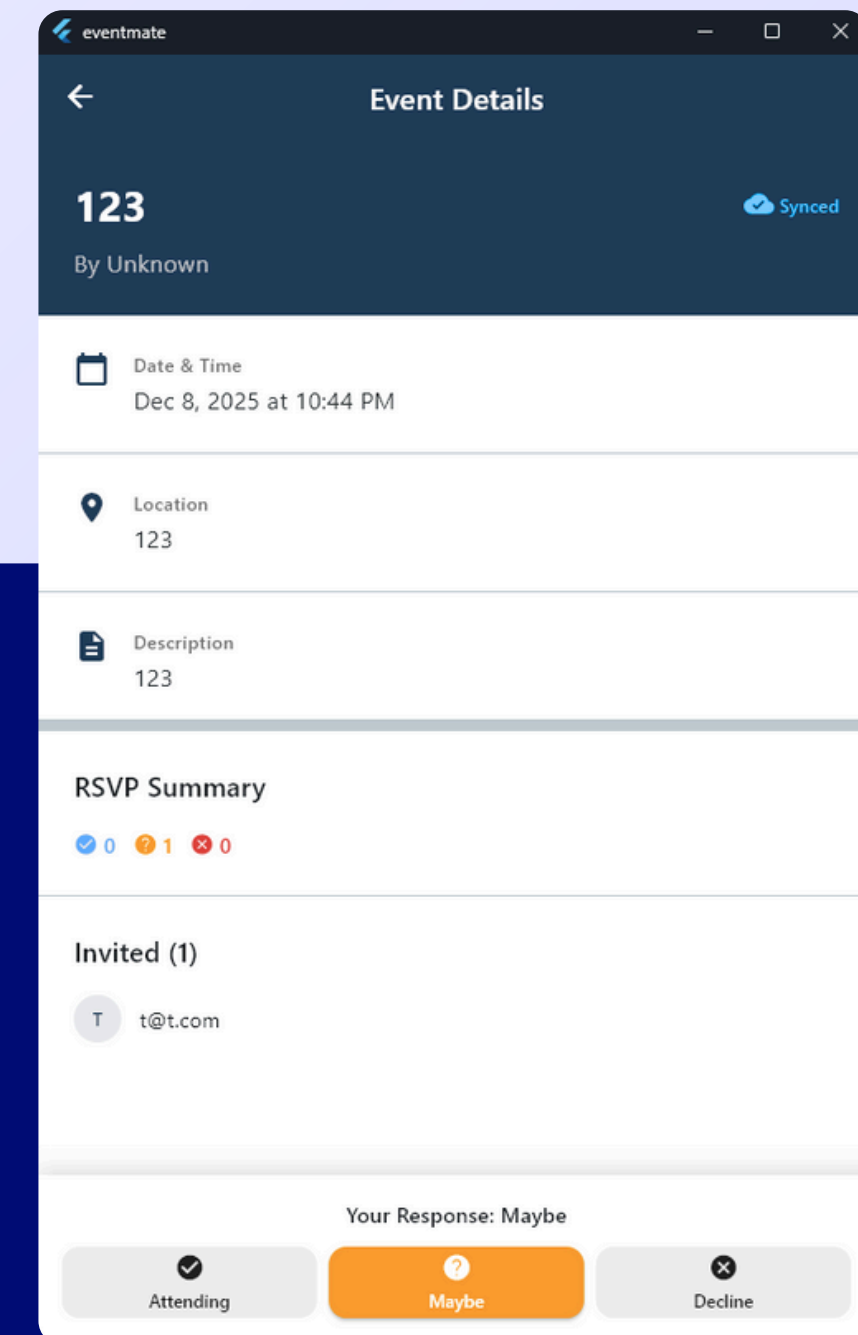
Event Details & RSVPs

Each event has a detail screen showing all the info plus RSVP counts. Invited users can respond with Going, Maybe, or Can't Go. Event creators can edit details, manage invites, or delete the event.



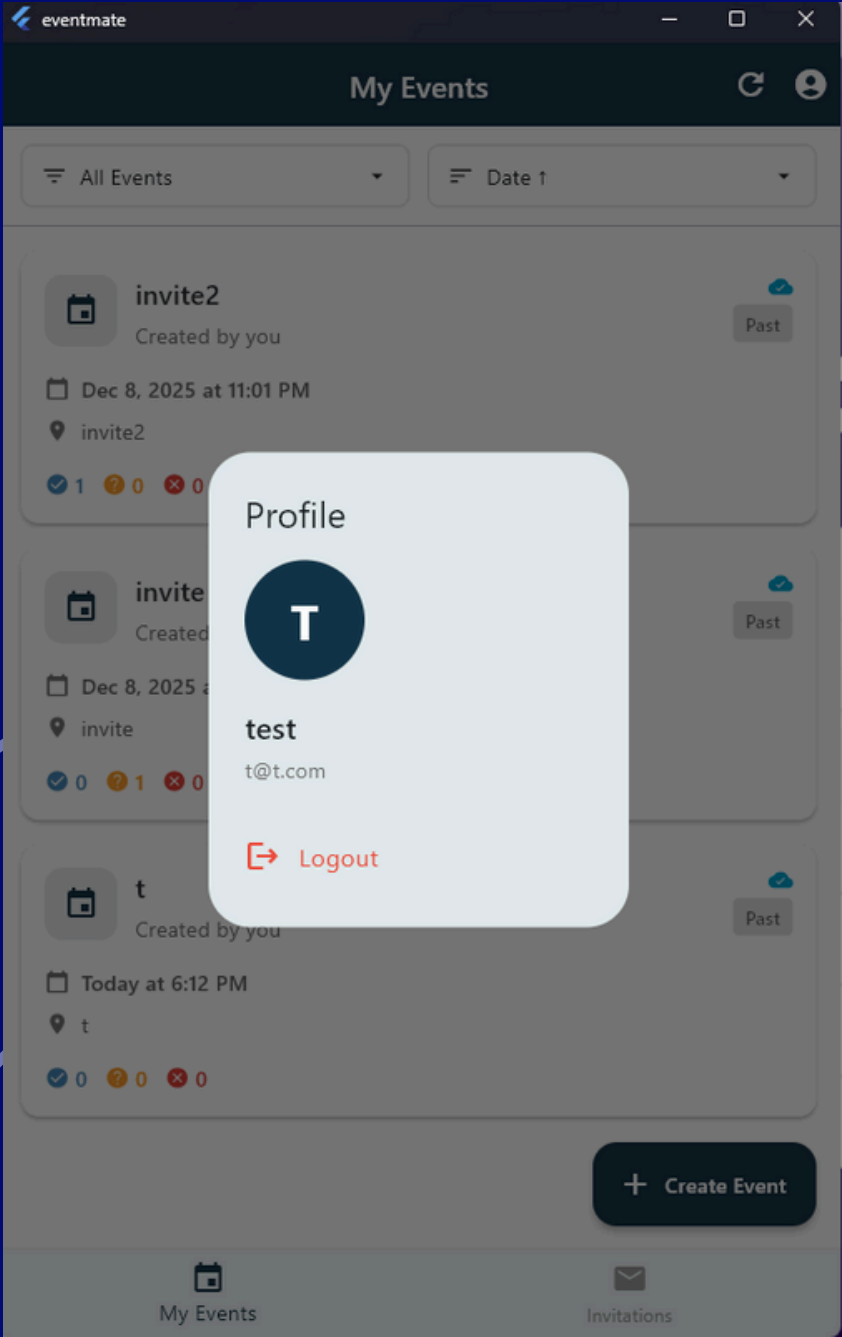
Offline Support

An orange banner appears when you're offline, but the app keeps working. You can browse your events, check details, and view cached profile info. When connectivity returns, everything syncs automatically.



Tech Stack

Layer	Technology
Frontend	Flutter
Backend	Firebase
Authentication	Firebase Auth
Database	Cloud Firestore
State Management	BLoC Pattern
Local Cache	Shared Preferences



Architecture

The app follows a clean layered structure. UI screens talk to BLoC components that manage state (authentication, sync status, connectivity). These connect to service classes that handle Firebase operations. Firestore's built-in offline persistence keeps data available without internet.

```
Stream<List<EventModel>> getMyEvents() {
  if (currentUserId == null) {
    return Stream.value([]);
  }

  return _firestore
    .collection('events')
    .where('creatorId', isEqualTo: currentUserId)
    .snapshots()
    .map((snapshot) {
      final events = snapshot.docs
        .map((doc) => _eventFromFirestore(doc))
        .toList();
      events.sort((a, b) => a.dateTime.compareTo(b.dateTime));
      return events;
    });
}

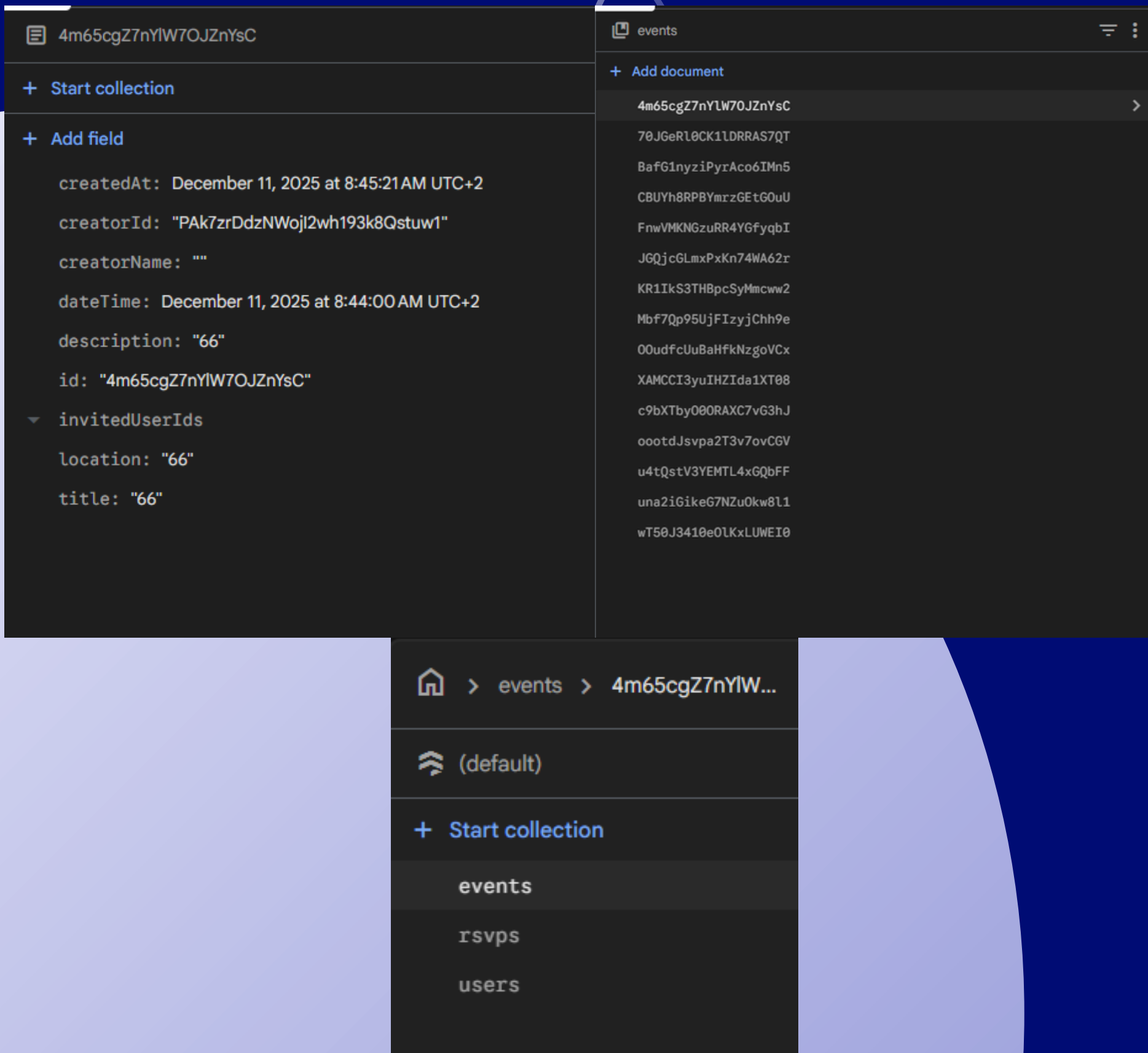
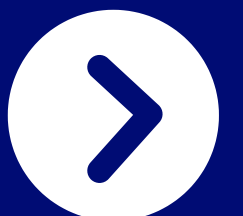
Stream<List<EventModel>> getInvitedEvents() {
  final userEmail = _auth.currentUser?.email;
  if (userEmail == null) {
    return Stream.value([]);
  }

  return _firestore
    .collection('events')
    .where('invitedUserIds', arrayContains: userEmail)
    .snapshots()
    .map((snapshot) {
      final events = snapshot.docs
        .map((doc) => _eventFromFirestore(doc))
        .toList();
      events.sort((a, b) => a.dateTime.compareTo(b.dateTime));
      return events;
    });
}

Future<EventModel?> getEvent(String eventId) async {
  try {
    final doc = await _firestore.collection('events').doc(eventId).get();
    if (doc.exists) {
      return _eventFromFirestore(doc);
    }
  }
  return null;
}
```

Database Structure

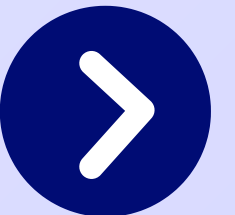
Three Firestore collections power the app. Users holds profile data. Events stores event details, creator ID, and invited emails. RSVPs tracks each user's response to each event. Real-time listeners keep everything in sync across devices.



Development Process

We follow a meticulous development process that includes ideation, design, development, testing, and deployment.

Each step is driven by a commitment to quality and innovation, ensuring that our apps are robust, scalable, and user-centric.



Summary

EventMate turns chaotic event planning into a smooth experience. Built with Flutter and Firebase, it offers real-time updates, secure authentication, and offline reliability. No more lost details or endless "who's coming?" messages.

Thank You

Questions?

Mobile and Embedded Computing

Puscasu Tudor & Duica Vlad

